

Techniques de compilation

Auditoire : CPI-2

Année Universitaire : 2025-2026

Code Classroom: **6xqrgwgl**

Dr.-Ing. Rakia Saidi

Maitre assistante en informatique
rakia.saidi@isimg.tn

Plan de ce cours

- **Chapitre 1**: Introduction à la compilation
- **Chapitre 2**: Rappels à la théorie des langages
- **Chapitre 3**: Analyse lexicale
- **Chapitre 4**: Analyse syntaxique
- **Chapitre 5**: Analyse sémantique
- **Chapitre 6**: Génération de code

Chapitre 2 : Initiation à la théorie des langages

Plan

1. Alphabets et langages
2. Expressions régulières
3. Automates
4. Grammaires hors contexte

Présentation générale

- La **théorie des langages** est la discipline qui vise l'automatisation du traitement des **langages**: savoir (sans intervention humaine) si un mot fait partie du champ lexical d'un langage donné, si une phrase est grammaticalement correcte par rapport à la grammaire du langage, etc.

1. Alphabets et langages

- **Alphabet**: un ensemble fini non vide de symboles, noté en général par Σ
 - Alphabet latin $\Sigma=\{a, b, c, \dots, z\}$
 - Alphabet arabe $\Sigma=\{\text{أ، ب، ت، ...، و}\}$
 - Alphabet binaire $\Sigma=\{0,1\}$
 - $\Sigma=\{\text{rouge, noir}, 0,1\}$
- **Mot** (lexème ou chaîne de caractère): une séquence de symboles dans un alphabet donné, noté ω
 - $\omega_1=\text{voiture}$, $\omega_2=\text{bonjour}$ $\rightarrow$ deux mots définis sur l'alphabet latin.
 - $\omega_1=00101$, $\omega_2=01$ $\rightarrow$ deux mots définis sur l'alphabet binaire
 - $\omega_1=\text{rouge}1$, $\omega_2=\text{noir}1$ $\rightarrow$ deux mots définis sur l'alphabet $\Sigma=\{\text{rouge, noir}, 0,1\}$
 - **Chaîne vide = mot vide** : notée ε , $|\varepsilon|=0$
- On note Σ^* : Ensemble infini contenant tous les mots de taille finie (taille 0, taille 1, taille 2, ...) possibles sur $\Sigma \rightarrow \varepsilon \in \Sigma^*$
- On note Σ^+ : Ensemble des mots non vides (de taille 1, 2, ...) possibles sur Σ
- On note Σ^k : Ensemble des mots de Σ^* de taille k

1. Alphabets et langages

- Un **langage** L défini sur un alphabet Σ est un ensemble de mots construits à partir de Σ^* et vérifiant une propriété P donnée.

$$L = \{w \in \Sigma^* \mid w \text{ possède la propriété } P\}$$

→ Un langage peut être infini mais il existe un nombre fini de symboles permettant de composer les mots de ce langage.

- Exemples: Soit l'alphabet $\Sigma = \{a, b, c\}$
 - Soit le langage $L1$ l'ensemble des mots de Σ^* ayant autant de a que du b
→ $L1 = \{c, cc, aabbc, \epsilon, abab, cab, caacbbc, \dots\}$
 - Soit le langage $L2$ l'ensemble des mots de Σ^* ayant exactement 4 a
→ $L2 = \{aaaa, bacaaa, ababaca, \dots\}$
 - Le langage vide $L = \emptyset$;
 - Le langage $\{\epsilon\}$ contenant le mot vide. Note: $\emptyset \neq \{\epsilon\}$

1. Alphabets et langages

Définition

- Un langage **régulier** L engendré sur un alphabet Σ est défini récursivement de la manière suivante:
 - $\{\epsilon\}$ est un langage régulier sur Σ
 - Si a est une lettre de Σ , alors $\{a\}$ est un langage régulier sur Σ
 - Si R est un langage régulier sur Σ , alors R^* et R^n sont des langages réguliers sur Σ
 - Si $R1$ et $R2$ sont des langages réguliers sur Σ , alors $R1 \cup R2$ et $R1R2$ sont des langages réguliers
- Les langages réguliers se décrivent facilement avec les expressions régulières

2. Expressions régulières

- Les expressions régulières sont une notation importante pour décrire les motifs de mots.
- Une **expression régulière (ER)** , dite aussi **expression rationnelle** sur un alphabet Σ se définit comme suit :
 - ε est une expression régulière
 - x est une expression régulière, pour tout $x \in \Sigma$
 - Si p et q sont deux expressions régulières alors $(p \mid q)$ est une expression régulière
 - Si p et q sont deux expressions régulières alors $(p.q)$ est une expression régulière
 - Si p est une expression régulière alors p^* est une expression régulière

2. Expressions régulières

- A chaque expression régulière, on associe un langage régulier de la façon suivante:
 - ϵ est une ER qui définit le langage $\{\epsilon\}$
 - Si $a \in \Sigma$, alors a est une ER qui définit le langage $\{a\}$
 - Soient X et Y deux ERs définissant les langages $L(X)$ et $L(Y)$, alors:
 - $(X)|(Y)$ est une ER définissant le langage $L(X) \cup L(Y)$
 - $(X)(Y)$ est une ER définissant le langage $L(X) L(Y)$
 - $(X)^*$ est une ER définissant le langage $(L(X))^*$
 - (X) est une ER définissant le langage $L(X)$
- Priorité $(*)$ > Priorité (concaténation) > priorité $(|)$

2. Expressions régulières

- Exemples: soit $\Sigma=\{a,b\}$
 - L'expression régulière $E_1=a|b$ dénote
 - L'expression régulière $E_2=(a|b)(a|b)$ dénote
 - L'expression régulière $E_3=(a|b)^*$ dénote
 - L'expression régulière $E_4=a|a^*b$ dénote
 - L'expression régulière $E_5=a^*b^*$ dénote
 - L'expression régulière $E_6=(ab)^*$ dénote

3. Automates

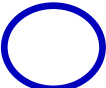
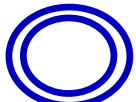
Définition d'un automate à états finis

- Un *automate à états finis* (A) est défini par un quintuplet $A=(\Sigma, E, e_0, T, \Delta)$:
 - Σ est l'**alphabet** des symboles d'entrée
 - E est l'**ensemble** fini d'états
 - $e_0 \in E$ est l'**état initial** de A
 - $T \subseteq E$ est un ensemble des **états finaux** (ou états terminaux)
 - Δ est une **fonction de transition** qui à tout couple formé d'un état et d'un symbole de Σ fait correspondre un ensemble éventuellement vide d'états ($\Delta(e_i, a)=\{e_{i1}, e_{i2}, \dots, e_{in}\}$).

3. Automates

Représentation d'un automate à états finis

- Un automate à états finis est représenté généralement par un graphe orienté étiqueté appelé *diagramme de transition*.
- Le diagramme est construit comme suit:
 - Pour chaque état $e_i \in E$ il existe un nœud étiqueté e_i ;
 - Pour chaque état $e_i \in E$ et chaque symbole $a \in \Sigma$ tels que $\Delta(e_i, a) = e_j$ il existe un arc étiqueté a entre le nœud e_i et le nœud e_j ;
- **Convention:** Pour la représentation graphique des automates, on utilise :

	Méthode 1	Méthode 2
Etat initial		
Etat intermédiaire		
Etat terminal		

3. Automates

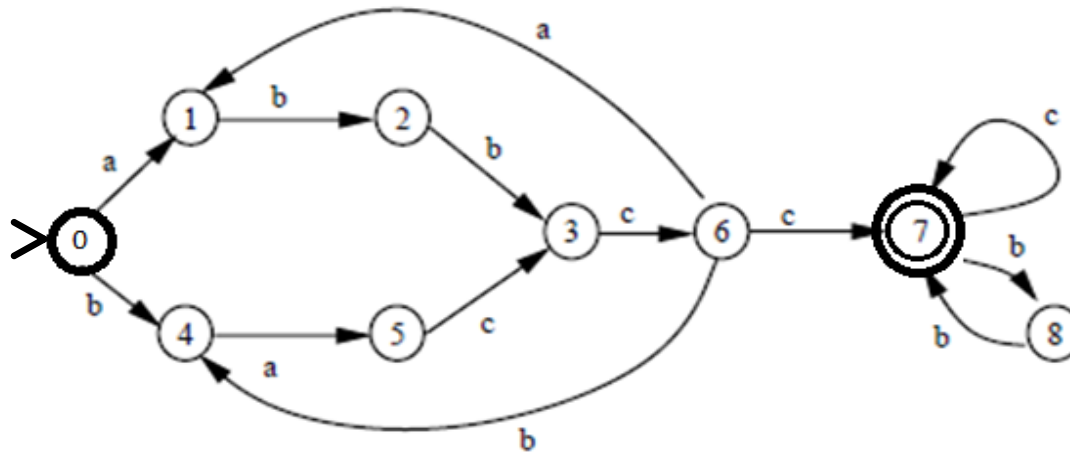
Représentation d'un automate à états finis

- Un automate à états finis est représenté par une table appelée *table de transition*.
- La table possède les caractéristiques suivantes:
 - Une ligne correspond à un état de l'automate;
 - Une colonne correspond à un symbole de l'alphabet.
 - Une case correspondant à la ligne e_i et un symbole a_j contient le résultat de $\Delta(e_i, a_j)$.
 - L'état initial est précédé par une flèche
 - Les états terminaux sont précédés chacun par une étoile

3. Automates

Représentation d'un automate à états finis

- Nous présentons ici un exemple d'automate :



→ Cet automate peut être décrit également par ce tableau :

Avec $E=\{0,1,2,3,4,5,6,7,8\}$, $e_0=0$, $T=\{7\}$

	a	b	c
→ 0	1	4	∅
1	∅	2	∅
2	∅	3	∅
3	∅	∅	6
4	5	∅	∅
5	∅	∅	3
6	1	4	7
7 *	∅	8	7
8	∅	7	∅

Table de transition

3. Automates

Exercice 1

- Construire les automates finis permettant d'accepter les langages suivants:
 1. $L(a^*)$ avec $\Sigma=\{a\}$
 2. $L((a|b)^*)$ avec $\Sigma=\{a,b\}$
 3. $L((ab)^*)$ avec $\Sigma=\{a,b\}$
 4. $L(a^*b^*)$ avec $\Sigma=\{a,b\}$
 5. $L(a(a|b)^*b)$
 6. $L((a|b|c)^*(a|c))$ avec $\Sigma=\{a,b,c\}$
 7. L'ensemble des mots comportant exactement 3 lettres avec $\Sigma=\{a,b,c\}$
 8. L'ensemble des mots comportant au plus 3 lettres avec $\Sigma=\{a,b,c\}$

3. Automates

Exercice 2

Soient les langages suivants:

$L_1 = \{w \in \{a,b,c\}^* \mid w \text{ commence par la lettre 'a'}\}$ $L_2 = \{w \in \{a,b,c\}^* \mid w \text{ se termine par la lettre 'a'}\}$

$L_3 = \{w \in \{a,b,c\}^* \mid w \text{ contient au moins une occurrence de la lettre 'a'}\}$ $L_4 = \{w \in \{a,b,c\}^* \mid w \text{ contient au moins 3 lettres dont la troisième lettre est obligatoirement 'a'}\}$

$L_5 = \{w \in \{a,b,c\}^* \mid \text{la troisième lettre à partir de la fin de } w \text{ est obligatoirement 'a'}\}$

- Donner l'expression régulière qui génère chacun de ces langages
- Construire l'automate à états finis qui permet de reconnaître chacun de ces langages

4. Grammaires hors contexte

Définition formelle

- Une grammaire est un quadruplet $G=(V_T, V_N, S, P)$ où:
 - V_T est un ensemble non vide de symboles terminaux (alphabet terminal)
 - V_N est un ensemble non vide de symboles non-terminaux (variables) avec $V_T \cap V_N = \emptyset$
 - S est un symbole initial appartenant à V_N appelé axiome ou *start symbol*
 - P est un ensemble fini de règles de production (règle de réécriture)
 - Une règle de production a la forme suivante $A \rightarrow \alpha$ où:
 - A est la variable définie par la règle, $(A \in (V_T \cup V_N)^+)$
 - $\alpha \in (V_T \cup V_N)^*$ est une chaîne composée de variables et de terminaux
 - Une règle de production précise que A peut être remplacée par la séquence α
- Les règles de production seront ordonnées, en respectant l'ordre des variables auxquelles elles s'appliquent. La première règle concerne toujours l'axiome.

4. Grammaires hors contexte

Définition formelle

- Exemple 2:
 - $V_T = \{\text{il, elle, parle, est, devient, court, reste, sympa, vite}\}$
 - $V_N = \{\text{PHRASE, SUJET, VERBE, COMPLEMENT, VERBEETAT, VERBEACTION}\}$
 - $S = \text{PHRASE}$
 - $P = \{\text{PHRASE} \rightarrow \text{SUJET VERBE COMPLEMENT}$
 $\text{SUJET} \rightarrow \text{il} | \text{elle}$
 $\text{VERBE} \rightarrow \text{VERBEETAT} | \text{VERBEACTION}$
 $\text{VERBEETAT} \rightarrow \text{est} | \text{devient} | \text{reste}$
 $\text{VERBEACTION} \rightarrow \text{court} | \text{parle}$
 $\text{COMPLEMENT} \rightarrow \text{sympa} | \text{vite}$
 $\}$

4. Grammaires hors contexte

Définition formelle

- Convention:

- Les symboles non-terminaux sont écrits en majuscules
- Les symboles terminaux sont écrits en minuscules
- Le mot vide est noté ε

- Exemple 1:

- Symboles terminaux (alphabet): $V_T = \{a, b\}$
- Symboles non-terminaux: $V_N = \{S\}$
- Axiome: S
- L'ensemble des règles de production P : $\begin{cases} S \rightarrow \varepsilon \\ S \rightarrow aSb \end{cases}$ qui se résument en $S \rightarrow \varepsilon | aSb$

4. Grammaires hors contexte

Arbre de dérivation

Définitions

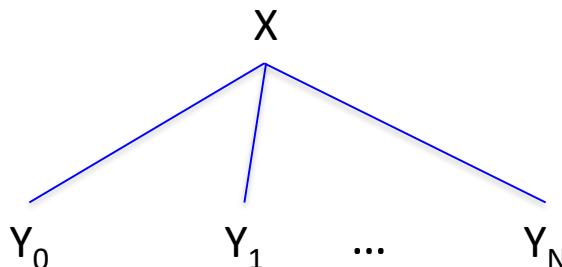
- On appelle dérivation l'application d'une ou plusieurs règles de production à partir d'un mot de $(V_T \cup V_N)^+$
- On notera:
 - « $\rightarrow$ » une dérivation obtenue par l'application d'une seule règle de production
 - « $\xrightarrow{*}$ » une dérivation obtenue par l'application de n règles de production, où $n > 1$.
- La grammaire de l'exemple 1:
 - $S \rightarrow \epsilon$
 - $S \rightarrow aSb$
 - $aSb \rightarrow aaSbb$
 - $S \xrightarrow{*} ab$ (2 règles)
 - $S \xrightarrow{*} aaabbb$ (4 règles)
 - $S \xrightarrow{*} aaSbb$ (2 règles)

4. Grammaires hors contexte

Arbre de dérivation

Définitions

- Pour déterminer si un mot w appartient ou non au langage défini par une grammaire G , on construit un arbre dont la racine est la variable de départ de G (l'axiome), les nœuds internes sont les symboles non-terminaux de G et les nœuds feuilles sont les symboles terminaux composant le mot w .
 - Les fils d'un nœud X sont $Y_0, Y_1, \dots, Y_N$ si et seulement si il existe une règle de production telle que $X \rightarrow Y_0 Y_1 \dots Y_N$.



→ Un tel arbre est appelé arbre de dérivation